

Maffe – A Lagrangian Approach to Maximum Agreement Forests

Anton Hoof ✉

Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany

Tobias Röhr ✉

Hasso Plattner Institute, University of Potsdam, Germany

Abstract

We describe Maffe, a solver for the maximum agreement forest (MAF) problem on rooted bifurcating phylogenetic trees. Maffe won all three tracks of the PACE 2026 Challenge (exact, heuristic, and lower bound). It combines reduction rules, a branch-and-price exact solver with a Lagrangian pricer, and a Lagrangian relaxation-based column generation heuristic for larger instances.

2012 ACM Subject Classification Theory of computation → Packing and covering problems; Theory of computation → Integer programming; Mathematics of computing → Combinatorial optimization; Applied computing → Operations research

Keywords and phrases maximum agreement forest, phylogenetic trees, integer programming, branch-and-price, column generation, lagrangian relaxation, heuristic algorithms

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Acknowledgements The implementation was assisted by OpenAI’s Codex with GPT 5.4 and GPT 5.5. We extensively tested the solver with fuzzing and STRIDE instances and take responsibility for its correctness.

1 Preliminaries and Notation

Let $T_1, \dots, T_k$ be rooted bifurcating phylogenetic trees on a common leaf set $\mathcal{L}$, and let $n = |\mathcal{L}|$. Each internal vertex has two children, and each leaf has a distinct label from $\mathcal{L}$. For a tree T , we write $V(T)$ and $E(T)$ for its vertex and edge sets, and use $V(T) \setminus \mathcal{L}$ for its internal vertices. For a non-empty vertex set $X \subseteq V(T)$, $\text{lca}(T, X)$ denotes the lowest common ancestor of X in T .

For non-empty $X \subseteq V(T)$, the Steiner tree $\text{ST}(T, X)$ is the unique minimal connected subgraph of T containing all vertices in X .

► **Definition 1** (Compatible leaf sets). *A non-empty leaf set $A \subseteq \mathcal{L}$ is compatible if $\text{ST}(T_1, A), \dots, \text{ST}(T_k, A)$ are equivalent as rooted, leaf-labeled trees after suppressing vertices of outdegree one.*

Thus, compatibility means that the trees induce the same branching structure on A . We write $\mathcal{C}$ for the family of all compatible leaf sets, and $\mathcal{C}' = \{A \in \mathcal{C} \mid |A| \geq 2\}$ for the non-singleton compatible sets.

► **Definition 2** (Agreement forest). *A partition $P \subseteq \mathcal{C}$ is a family of pairwise disjoint compatible sets whose union is $\mathcal{L}$. Such a partition is an agreement forest if for every tree T_i , the Steiner trees $\text{ST}(T_i, A)$ with $A \in P$ are pairwise vertex-disjoint.*

The maximum agreement forest (MAF) problem asks for an agreement forest with the minimum number of components.

Despite the name, the objective is to minimize the component count: fewer components preserve more of the common tree structure.



© Anton Hoof and Tobias Röhr;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:11

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2 Reduction Rules

We use reductions that either replace a fixed part of the final forest by a smaller representative, mark edges that must be cut, or split the instance into subinstances whose solutions can be expanded back to the original leaf set and combined. The multi-tree setting follows Shi et al. [8]. The common subtree and common chain reductions, including the truncation length for multiple binary trees, follow Kelk et al. [4]. Cluster partitioning follows Li and Zeh [5], and the two-tree 3-2 rule is based on the rooted kernel of Kelk, Linz, and Meuwese [3]. We summarize the reductions below. See these papers for their formal statements and correctness proofs.

■ **Cherry reduction.** Common pendant subtrees are compressed to a representative leaf, which is expanded again to the whole component in the final solution.

■ **Chain reduction.** A common chain is shortened to $k + 1$ leaves, where k is the number of input trees.

■ **3-2 reduction.** For two-tree instances, a cherry in one tree whose two leaves have exactly one hanging subtree attached to their connecting path in the other tree forces the edge to that hanging subtree to be cut.

■ **5-3 reduction (new).** The implementation also uses the analogous triple case: if a rooted triple in one tree forces two hanging subtrees in the other tree to be separated, both corresponding edges are cut.

■ **Generalized cluster reduction (new).** After the 3-2 and 5-3 reductions, the input may consist of forests. We call a leaf set C a *cluster* if, in each forest, at most one additional cut separates C from all remaining leaves.

We choose the smallest such cluster and solve the instance restricted to C twice: with and without a dummy leaf representing the rest of the instance.

If the solution with the dummy leaf has fewer components, we replace C in the remaining instance by a representative leaf. Otherwise, we remove C without adding a representative.

3 Two-Tree Branch-and-Price

All our two-tree solvers use a modified version of the exponential-size ILP formulation of Olver et al. [7].

Set-packing formulation.

For every non-singleton compatible set $A \in \mathcal{C}'$, we introduce a binary variable x_A indicating whether A is selected as a component of the forest. Singleton components are implicit: any leaf not covered by a selected non-singleton component becomes a singleton component. If a selected component A has size $|A|$, then it replaces $|A|$ singleton components by one component and therefore saves $|A| - 1$ components. Minimizing the final number of components is thus

77 equivalent to the following weighted set packing problem:

$$\begin{aligned}
78 \quad & \min \quad n - \sum_{A \in \mathcal{C}'} x_A (|A| - 1) \\
79 \quad & \text{s.t.} \quad \sum_{A \in \mathcal{C}': v \in \text{ST}(T_1, A)} x_A \leq 1 & \forall v \in V(T_1) \setminus \mathcal{L}, & \text{(P1)} \\
80 \quad & \sum_{A \in \mathcal{C}': v \in \text{ST}(T_2, A)} x_A \leq 1 & \forall v \in V(T_2) \setminus \mathcal{L}, & \text{(P2)} \\
81 \quad & x_A \in \{0, 1\} & \forall A \in \mathcal{C}'.
\end{aligned}$$

82 The packing constraints P1 and P2 enforce that selected components have internally disjoint
83 Steiner trees in both T_1 and T_2 . They also imply that a leaf is covered by at most one
84 selected non-singleton component: if $\ell \in A$ and $|A| \geq 2$, then the parent of ℓ in T_1 and in T_2
85 lies in the corresponding Steiner tree.

86 Column generation.

87 The ILP formulation has a tight LP relaxation in practice, as observed by Olver et al. [7] and
88 Frohn et al. for the unrooted version [1]. Since there can be exponentially many variables, we
89 solve the LP relaxation using column generation. We start with a subset of components and
90 solve the resulting restricted LP. A pricing algorithm then searches for omitted components
91 with negative reduced cost, whose variables could improve the LP objective. We add these
92 variables as columns and repeat until pricing certifies that none remain.

93 Frohn et al. solve the pricing problem by finding a single component with the most
94 negative reduced cost. This is equivalent to the rooted weighted maximum agreement subtree
95 (rWMAST) problem, where one asks for a compatible leaf set of maximum total weight. For
96 two trees, rWMAST can be solved by standard dynamic programming in $\mathcal{O}(n^2)$ time [1].
97 However, in our experiments, the rWMAST pricer led to slow LP convergence because it
98 returns only one component per iteration and the LP suffers from dual degeneracy. Maffe
99 instead solves a partition problem derived from a Lagrangian relaxation, allowing the pricer
100 to return several components with negative reduced cost per iteration.

101 3.1 Lagrangian Relaxation

102 We first describe the relaxation without fixed edges. Section 3.2 incorporates reductions and
103 branching decisions. The relaxation keeps components disjoint in T_1 but allows them to
104 overlap in T_2 , charging for the internal vertices they use there. This retains enough structure
105 to find several components together by dynamic programming. Hence, we dualize the set
106 packing constraints P2 associated with the internal vertices of T_2 . This yields the Lagrangian
107 functional

$$\begin{aligned}
108 \quad & F(\lambda) = \min_x \quad n - \sum_{A \in \mathcal{C}'} x_A (|A| - 1) \\
109 \quad & \quad + \sum_{v \in V(T_2) \setminus \mathcal{L}} \lambda_v \left(-1 + \sum_{A \in \mathcal{C}': v \in \text{ST}(T_2, A)} x_A \right) \\
110 \quad & \text{s.t.} \quad \sum_{A \in \mathcal{C}': v \in \text{ST}(T_1, A)} x_A \leq 1 & \forall v \in V(T_1) \setminus \mathcal{L}, & \text{(P1)} \\
111 \quad & x_A \in \{0, 1\} & \forall A \in \mathcal{C}',
\end{aligned}$$

For any $\lambda \geq 0$, $F(\lambda)$ is a lower bound on the minimum number of components: the added term is non-positive for every feasible agreement forest. The Lagrangian dual $\max_{\lambda \geq 0} F(\lambda)$ seeks the strongest such bound.

The partition subproblem.

For fixed multipliers λ , the net saving from selecting a non-singleton component $A \in \mathcal{C}'$ is

$$w_\lambda(A) = |A| - 1 - \sum_{v \in \text{ST}(T_2, A) \setminus \mathcal{L}} \lambda_v.$$

Apart from the constant $n - \sum_v \lambda_v$, evaluating $F(\lambda)$ amounts to maximizing the sum of these savings subject to the packing constraints in T_1 . Adding uncovered leaves as singleton components gives a partition P of $\mathcal{L}$. Since $\sum_{A \in P} |A| = n$, the equivalent partition objective assigns each component weight -1 minus its vertex charges in T_2 . We call this the rooted weighted maximum agreement subtree partition (rWMASTP) problem. Its components must be compatible with both trees and disjoint in T_1 . Overlaps in T_2 are allowed and charged through the weights. In Appendix A, we describe a dynamic programming solution for two trees in time $\mathcal{O}(n^2)$.

3.2 Branching and Pricing

Edge branching.

We branch on whether an input-tree edge belongs to the Steiner tree of a selected component. A cut edge must be unused; a forced edge must be used by exactly one component. For a current fractional LP solution x^* , define the usage of an edge $e \in E(T_j)$ as $y_e^j = \sum_{A \in \mathcal{C}': e \in E(\text{ST}(T_j, A))} x_A^*$. In any integral agreement forest, $y_e^j \in \{0, 1\}$. If $0 < y_e^j < 1$, we create two children: one where e is cut and one where e is forced to be used by exactly one selected component. The two children impose $y_e^j = 0$ and $y_e^j = 1$, respectively.

Let $E_{\text{cut}}^1 \subseteq E(T_1)$ and $E_{\text{cut}}^2 \subseteq E(T_2)$ be the cut edges, and let $E_{\text{forced}}^1 \subseteq E(T_1)$ and $E_{\text{forced}}^2 \subseteq E(T_2)$ be the forced edges, which arise from reductions and branching decisions.

Branching constraints in the master LP.

Cut edges exclude any component whose Steiner tree uses them. Forced edges are handled by equalities in the master LP and dualized in both trees during pricing. The DP does not enforce forced edges as hard constraints: a pricing partition may leave them unused or, in T_2 , use them more than once. The master LP enforces their final usage.

► **Definition 3** (Components and partitions for pricing). *Let*

$$\mathcal{C}_E = \{A \in \mathcal{C} \mid E(\text{ST}(T_j, A)) \cap E_{\text{cut}}^j = \emptyset \text{ for } j \in \{1, 2\}\}, \quad \mathcal{C}'_E = \mathcal{C}_E \cap \mathcal{C}'.$$

Let $\mathcal{P}_E$ be the family of partitions of $\mathcal{L}$ into components from $\mathcal{C}_E$ whose Steiner trees are internally disjoint in T_1 . No forced-edge coverage requirements are imposed on $\mathcal{P}_E$.

In particular, the all-singleton partition belongs to $\mathcal{P}_E$ at every search node. The master LP uses variables indexed by $\mathcal{C}'_E$ and retains the forced-edge equalities

$$\sum_{A \in \mathcal{C}'_E : e \in E(\text{ST}(T_j, A))} x_A = 1 \quad \forall j \in \{1, 2\}, e \in E_{\text{forced}}^j. \quad (\text{F})$$

Reduced costs and partition weights.

We also use the partition DP to solve the pricing problem. For this purpose, we choose component weights so that maximizing the partition weight is equivalent to minimizing the sum of the reduced costs of its non-singleton components.

Let $w_v^j \geq 0$ denote the packing duals and $w_e^j \in \mathbb{R}$ the forced-edge duals, using a sign convention in which they enter reduced costs with a plus sign. For $A \in \mathcal{C}'_E$, the reduced cost is

$$\bar{c}_A = 1 - |A| + \sum_{j=1}^2 \sum_{v \in \text{ST}(T_j, A) \setminus \mathcal{L}} w_v^j + \sum_{j=1}^2 \sum_{e \in E(\text{ST}(T_j, A)) \cap E_{\text{forced}}^j} w_e^j.$$

To obtain these weights, assign each internal vertex in both trees the negative of its packing dual and each forced edge the negative of its equality dual. All other edges have weight zero. With the component penalty of -1 from Definition 5, the resulting component weight is

$$w(A) = -|A| - \bar{c}_A \quad \text{for } A \in \mathcal{C}'_E, \quad w(\{\ell\}) = -1.$$

Since every partition covers exactly n leaves, the terms $-|A|$ sum to the constant $-n$. Maximizing total weight therefore minimizes the sum of the non-singleton reduced costs. The DP maximizes $\sum_{A \in P} w(A)$ over $P \in \mathcal{P}_E$, and the pricer returns every non-singleton component of an optimal partition with negative reduced cost.

► **Lemma 4** (Pricing completeness). *There exists a component $A \in \mathcal{C}'_E$ with negative reduced cost, including the dual contributions of the current forced-edge constraints, if and only if the partition pricer returns at least one such component.*

Proof. For every $P \in \mathcal{P}_E$, the identity $\sum_{A \in P} |A| = n$ gives

$$\sum_{A \in P} w(A) = -n - \sum_{A \in P \cap \mathcal{C}'_E} \bar{c}_A.$$

The all-singleton partition has weight $-n$. If $A \in \mathcal{C}'_E$ has $\bar{c}_A < 0$, form a partition from A and all remaining leaves as singletons. This partition respects the cut edges and is internally disjoint in T_1 . It belongs to $\mathcal{P}_E$ because forced-edge equalities are dualized, not enforced by the DP. Its weight is $-n - \bar{c}_A > -n$. Consequently, an optimal pricing partition also has weight greater than $-n$ and must contain at least one component with negative reduced cost. Conversely, every component returned by the pricer belongs to $\mathcal{C}'_E$ and passes the negative reduced-cost test. ◀

Thus, if the pricer returns no component, the current restricted LP solution is optimal for the full node LP. This certification applies after solving a feasible restricted LP. Infeasible restricted LPs require Farkas pricing.

4 Multiple Tree Compact ILP

The set-packing formulation above also extends to $k \geq 3$ trees by adding packing constraints for every tree. For reduced multi-tree instances that meet the size and edge-state conditions in Section 6, we use a compact ILP instead of branch-and-price. SCIP can apply its general MIP cuts to the compact model, which strengthens the LP relaxation. For this back end, we use the compact ILP formulation that is equivalent to the set-packing formulation given in [7]. Olver et al. describe the compact ILP for two trees, and we apply the construction to all k trees.

188 **4.1 Formulation**189 **States and transitions.**

190 Assume that the leaves are numbered $1, \dots, n$. Let $\mathcal{Z} = \{(a, b) \mid 1 \leq a \leq b \leq n\}$ be the
 191 set of states and let $\mathcal{Z}_{\mathcal{L}} = \{(a, a) \mid a \in \mathcal{L}\}$ be the leaf states. For a state $r = (a, b)$, write
 192 $m_t(r) = \text{lca}(T_t, \{a, b\})$. We construct a directed acyclic graph with left arcs U_L and right arcs
 193 U_R . For states $r = (a, b)$ and $s = (c, d)$, the arc (r, s) is present if $m_t(s)$ is a strict descendant
 194 of $m_t(r)$ in every tree T_t . It is a left arc if $a = c$, and a right arc if $b = c$. Let $U = U_L \cup U_R$.
 195 A state records a possible branching point through a pair of representative leaves. Left and
 196 right arcs connect this branching point to the two branches of a component. The variable
 197 y_a records whether a transition arc a is selected. For a state r , write $L(r) = \delta^+(r) \cap U_L$,
 198 $R(r) = \delta^+(r) \cap U_R$, and $I(r) = \delta^-(r)$. Write $[k] = \{1, \dots, k\}$ and $\mathcal{I}_t = V(T_t) \setminus \mathcal{L}$. For a set
 199 A of arcs, write $y(A) = \sum_{a \in A} y_a$.

200 **Flow and component count.**

201 The balance constraint gives each selected internal state equally many left and right transitions.
 202 The flow constraint allows a component to start at a state, but prevents incoming flow from
 203 ending at an internal state. Thus $y(L(r)) - y(I(r))$ counts components starting at r , while
 204 $x_{\{\ell\}}$ records whether leaf ℓ is a singleton. Using the convention of Olver et al., the objective
 205 value below is the number of components minus one. The compact LP relaxation is

$$\begin{aligned}
 206 \quad & \min \quad \sum_{r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}} (y(L(r)) - y(I(r))) + \sum_{\ell \in \mathcal{L}} x_{\{\ell\}} - 1 \\
 207 \quad & \text{s.t.} \quad y(L(r)) = y(R(r)) & \forall r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}, & \text{(balance)} \\
 208 \quad & y(L(r)) \geq y(I(r)) & \forall r \in \mathcal{Z} \setminus \mathcal{Z}_{\mathcal{L}}, & \text{(flow)} \\
 209 \quad & x_{\{\ell\}} = 1 - y(I((\ell, \ell))) & \forall \ell \in \mathcal{L}, & \text{(leaf)} \\
 210 \quad & \sum_{a \in \Gamma_t(v)} y_a \leq 1 & \forall t \in [k], v \in \mathcal{I}_t, & \text{(capacity)} \\
 211 \quad & x_{\{\ell\}} \geq 0 & \forall \ell \in \mathcal{L}, \\
 212 \quad & y_a \geq 0 & \forall a \in U.
 \end{aligned}$$

213 **Vertex capacities.**

214 Here $\Gamma_t(v)$ is the set of compact arcs whose corresponding transition uses the internal vertex
 215 v in T_t . More explicitly, for an arc $a = (r, s)$, the transition uses the vertices on the path from
 216 $m_t(r)$ to $m_t(s)$, counting $m_t(r)$ only for left arcs and not counting $m_t(s)$ if s is an internal
 217 state. The fourth constraint is the additional capacity constraint missing from the printed
 218 compact formulation of Olver et al. Without it, the compact flow can select components
 219 whose Steiner trees overlap away from their roots. The compact ILP is obtained by requiring
 220 the arc variables y_a and singleton variables $x_{\{\ell\}}$ to be integral.

221 The compact state space also suggests a way to derive cuts for the multi-tree exponential
 222 formulation. Some valid inequalities over compact arcs lift to cuts over compatible components.
 223 If the pricer can evaluate the resulting component coefficient, these cuts remain compatible
 224 with column generation.

4.2 Reducing the Model Size

The formulation has $\mathcal{O}(n^2)$ states and $\mathcal{O}(n^3)$ compact arcs, and therefore already contains $\mathcal{O}(n^3)$ transition variables before the capacity constraints are considered. Building all these variables is too expensive for the instances targeted by the compact solver.

The implementation reduces the model size in three ways.

Lazy capacity constraints.

The capacity constraints are handled lazily: the solver starts without all rows of type (capacity), computes the capacity usage of the current solution, and adds only violated capacity rows. Before the compact ILP is solved, we run the root relaxation of the branch-and-price model by column generation. The resulting columns are used to compute an initial incumbent for the compact model. We also add the capacity rows that are needed for this LP certificate, namely rows with relevant dual value or support in the root relaxation, before the lazy capacity separator is enabled.

Merging equivalent states.

Equivalent states are merged. The mathematical description above indexes states by leaf pairs. In the implementation, two states r and s are identified if

$$(m_1(r), \dots, m_k(r)) = (m_1(s), \dots, m_k(s)).$$

All tests for valid transitions, all flow coefficients, and all capacity usages depend on a state only through this tuple of lowest common ancestors. Identifying such states therefore only removes duplicate copies of the same compact-flow state. After the state merge, parallel transition arcs with the same source, target, and orientation are also represented by a single variable.

Folding transition bicliques.

For one fixed orientation, suppose the transition graph contains all arcs $S \times Q$ between a set S of source states and a set Q of target states. Choosing arcs inside this complete bipartite graph only has to decide which sources are used and how many arcs enter each target. The implementation therefore replaces the variables $y_{s,q}$ by source variables $p_s \in \{0, 1\}$ for $s \in S$ and target-count variables $z_q \in \mathbb{Z}_{\geq 0}$ for $q \in Q$, together with the row $\sum_{s \in S} p_s = \sum_{q \in Q} z_q$. The source-side objective and flow coefficients are assigned to the variables p_s , while the target-side flow coefficients are assigned to the variables z_q . Bicliques are folded only when every affected capacity row can be charged consistently to one of these two sides. Any original integral solution projects to these counts, and any integral folded solution can be expanded by matching selected sources to target counts inside $S \times Q$. This preserves the objective value and every retained or lazily generated capacity row.

5 Heuristic Solver

The heuristic solver first applies the same reductions as the exact solver, but the exact solves used inside cluster reductions have a time limit and use the best incumbent found within that limit. After reductions, the remaining instance size determines the next step. For small and medium-size instances, the branch-and-price (B&P) solver is run and returns the best incumbent if the time limit is reached. For larger instances, solving the full LP is too expensive, so we use a two-stage set-packing heuristic.

266 **Stage 1: generating components.**

267 We optimize the Lagrangian dual from Section 3.1 using a bundle method [2] and record the
 268 components found while evaluating the pricing subproblems. Since the generated components
 269 become less diverse once the dual values stabilize, we also evaluate the Lagrangian functional
 270 at random perturbations of the best dual values found so far. The same process is run once
 271 with the packing constraints of T_2 dualized and once with the roles of T_1 and T_2 exchanged.

272 **Stage 2: selecting components.**

273 We solve restricted set-packing ILPs over the generated components. Components are
 274 ranked by how often they were generated, with component size and a random value used
 275 as tiebreakers. The heuristic then considers successively longer prefixes of this ranking. If
 276 time remains after the final restricted ILP, the incumbent is used as a warm start for a final
 277 branch-and-price run.

278 **6 Implementation Details**

279 The implementation uses SCIP Optimization Suite 10.0.2 as its mixed-integer programming
 280 (MIP) framework. SCIP uses HiGHS 1.15.1 to solve its LP relaxations.

281 **6.1 Exact Track**282 **Choosing the solver.**

283 After reductions, the exact track chooses between exact back ends according to the residual
 284 instance. Very small residual instances are solved directly. Multi-tree residual instances with
 285 $k \geq 3$, fewer than 500 leaves, and no fixed edge states use the compact ILP formulation from
 286 Section 4. All remaining residual instances use a SCIP branch-and-price algorithm over the
 287 set-packing formulation.

288 **Pricing and node bounds.**

289 In the branch-and-price back end, each search node maintains the cut and forced edge sets
 290 introduced above. At a node, the restricted LP is solved, the rWMASTP pricer is called with
 291 the current dual values, and all generated components with negative reduced cost are added
 292 as columns. If the restricted LP is infeasible, Farkas pricing uses an infeasibility certificate to
 293 search for columns that could restore feasibility. If no such column exists, the full node LP is
 294 infeasible and the node can be pruned. By Lemma 4, the pricing loop terminates when no
 295 component with negative reduced cost remains. The resulting full node LP value provides a
 296 lower bound for the search node.

297 Branching decisions fix edges either as cut edges or as forced edges. Cut edges restrict
 298 the DP transitions; forced-edge equalities in both trees contribute dual weights to the pricing
 299 objective, as described in Section 3.2.

300 **Primal heuristics and stabilization.**

301 The solver disables SCIP's default separation and instead relies on the structure of the packing
 302 formulation, the custom branching decisions, and a sub-MIP heuristic over the current column
 303 pool. These sub-MIPs are solved with HiGHS. To stabilize the LP convergence, the branch-
 304 and-price back end uses box-style dual stabilization [6] with weights 0.75, 0.50, and 0.25

and update weight 0.25. At the root node, the pricer is also called twice with a dual vector shifted halfway from the current dual solution toward the stabilized dual reference point. These additional root pricing calls add columns before they become negative at the current restricted master solution.

6.2 Lower Bound Track

The lower-bound track is the same as the exact track, except that the sub-MIP heuristic is disabled and the solver stops early once the bound meets its stopping criterion.

6.3 Heuristic Track

The heuristic track uses the reductions from Section 2. Cluster-reduction subsolves use the same exact solver with a time limit of $\max\left(\frac{\#\text{cluster leaves}}{\#\text{total leaves}} \cdot 300, 0.25\right)$ seconds and lift the best incumbent found within that limit. If at most 1000 leaves remain after reductions, the heuristic track continues with the B&P solver and returns the best incumbent when time expires.

For larger reduced instances, the Lagrangian multipliers are optimized with the Conic Bundle optimizer [2]. The main column-generation phase uses at most 15% of the time remaining after reductions, the perturbation phase uses 5%, and both directions together use at most 40%. The restricted set-packing ILPs are solved over the best 3000, 7000, and 12000 generated components, with time limits of 10 seconds, 30 seconds, and the remaining solve time, respectively.

References

- 1 Martin Frohn, Steven Kelk, and Simona Vychytilova. A branch-&-price approach to the unrooted maximum agreement forest problem. *Oper. Res. Lett.*, 63:107364, 2025. URL: <https://doi.org/10.1016/j.orl.2025.107364>, doi:10.1016/J.ORL.2025.107364.
- 2 Christoph Helmborg. ConicBundle: A callable library for convex optimization, 2021. TU Chemnitz. Available at: https://www.tu-chemnitz.de/mathematik/discrete_math/helmborg/ConicBundle/.
- 3 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear kernel for the rooted subtree prune and regraft distance. *Inf. Process. Lett.*, 180:106336, 2023. URL: <https://doi.org/10.1016/j.ipl.2022.106336>, doi:10.1016/J.IPL.2022.106336.
- 4 Steven Kelk, Ruben Meuwese, and Leo van Iersel. A kernel for the maximum agreement forest problem on multiple binary phylogenetic trees, 2026. URL: <https://arxiv.org/abs/2602.22281>, arXiv:2602.22281, doi:10.48550/arXiv.2602.22281.
- 5 Zhijiang Li and Norbert Zeh. Computing maximum agreement forests without cluster partitioning is folly. In Kirk Pruhs and Christian Sohler, editors, *25th Annual European Symposium on Algorithms, ESA 2017, Vienna, Austria, September 4-6, 2017*, volume 87 of *LIPIcs*, pages 56:1–56:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017. URL: <https://doi.org/10.4230/LIPIcs.ESA.2017.56>, doi:10.4230/LIPICS.ESA.2017.56.
- 6 Roy E. Marsten, W. W. Hogan, and J. W. Blankenship. The boxstep method for large-scale optimization. *Oper. Res.*, 23(3):389–405, 1975. URL: <https://doi.org/10.1287/opre.23.3.389>, doi:10.1287/OPRE.23.3.389.
- 7 Neil Olver, Frans Schalekamp, Suzanne van der Ster, Leen Stougie, and Anke van Zuylen. A duality based 2-approximation algorithm for maximum agreement forest. *Math. Program.*, 198(1):811–853, 2023. URL: <https://doi.org/10.1007/s10107-022-01790-y>, doi:10.1007/S10107-022-01790-Y.

8 Feng Shi, Jianer Chen, Qilong Feng, Xiaojun Ding, and Jianxin Wang. Algorithms for maximum agreement forest of multiple general trees. *CoRR*, abs/1411.0062, 2014. URL: <http://arxiv.org/abs/1411.0062>, arXiv:1411.0062.

A Rooted Weighted Maximum Agreement Subtree Partition

► **Definition 5** (Rooted weighted maximum agreement subtree partition). *The rooted weighted maximum agreement subtree partition (rWMASTP) problem asks for a partition $P \subseteq \mathcal{C}$ of $\mathcal{L}$ into compatible sets such that the Steiner trees of the blocks are internally disjoint in T_1 and the total weight is maximized. Given weights w_v on internal vertices $v \in \left(\bigcup_{t=1}^k V(T_t)\right) \setminus \mathcal{L}$ and weights w_e on edges $e \in \bigcup_{t=1}^k E(T_t)$, each component incurs a penalty of -1 . The weight of a compatible component $A \in \mathcal{C}$ is*

$$w(A) = -1 + \sum_{t=1}^k \left(\sum_{v \in \text{ST}(T_t, A) \setminus \mathcal{L}} w_v + \sum_{e \in E(\text{ST}(T_t, A))} w_e \right).$$

For the Lagrangian evaluation in Section 3.1, only vertices in T_2 receive non-zero weights. For pricing in Section 3.2, vertices and forced edges in both trees receive the negative of their corresponding LP duals. Cut edges restrict the feasible partitions as in Definition 3.

Integer programming formulation.

If we associate with each compatible component $A \in \mathcal{C}$ a binary variable x_A indicating whether A is selected, we obtain the following integer programming formulation:

$$\begin{aligned} \max \quad & \sum_{A \in \mathcal{C}} w(A) x_A \\ \text{s.t.} \quad & \sum_{A \in \mathcal{C}: \ell \in A} x_A = 1 & \forall \ell \in \mathcal{L}, \\ & \sum_{A \in \mathcal{C}: v \in \text{ST}(T_1, A)} x_A \leq 1 & \forall v \in V(T_1) \setminus \mathcal{L}, \\ & x_A \in \{0, 1\} & \forall A \in \mathcal{C}. \end{aligned}$$

A.1 Dynamic Programming for Two Trees

To solve the rWMASTP pricing subproblem efficiently on two trees T_1 and T_2 , we use a dynamic programming (DP) algorithm that sweeps bottom-up on both trees. The DP is inspired by the rooted weighted maximum agreement subtree (rWMAST) DP algorithm presented in [1].

State space.

For a vertex $u \in V(T)$, write $T|_u$ for the subtree rooted at u , and write $\mathcal{L}(T|_u)$ for the leaves below u . The DP uses pairs of vertices $(u, v) \in V(T_1) \times V(T_2)$ whose leaf sets intersect:

$$\mathcal{S} = \{(u, v) \in V(T_1) \times V(T_2) \mid \mathcal{L}(T_1|_u) \cap \mathcal{L}(T_2|_v) \neq \emptyset\}.$$

For two trees, the size of this state space is bounded by $\mathcal{O}(n^2)$.

Open and closed components.

For each state $(u, v) \in \mathcal{S}$, the DP computes two open state values representing the maximum total weight of a partition of the leaves $\mathcal{L}(T_1|_u)$ into compatible components, where all components are closed except for one active component $A \subseteq \mathcal{L}(T_1|_u)$ that is open at u in T_1 and at v in T_2 . A closed component is complete and cannot acquire more leaves, whereas the active component may still grow. The active component's Steiner tree is extended to include the current vertices u and v , taking the form $\text{ST}(T_1, A \cup \{u\})$ in T_1 and $\text{ST}(T_2, A \cup \{v\})$ in T_2 . The active component is the only component that can be extended above u in T_1 . Open-state values include the weights of these extensions and the penalties of closed components, but defer the active component's -1 penalty until it is closed.

- $O(u, v)$: The maximum weight of a partition of $\mathcal{L}(T_1|_u)$ containing one such active component A , which is connected to u and v respectively.
- $O_{\text{close}}(u, v)$: The maximum weight of a partition of $\mathcal{L}(T_1|_u)$ where u and v are the lowest common ancestors of the active component A in T_1 and T_2 , respectively, making it eligible to be closed.

The DP also computes a value for partitions with no active component:

- $C(u)$: The maximum total weight of a partition of $\mathcal{L}(T_1|_u)$ where all components are closed.

Transitions and solution recovery.

The transitions propagate these values bottom-up: open states are initialized at matching leaves (ℓ, ℓ) , extended upwards along edges in both trees, combined at matching split nodes to transition to closable states, and closed at their lowest common ancestors to transition to the closed forest state $C(u)$. For each internal node $u \in V(T_1) \setminus \mathcal{L}$ with children u_L and u_R :

$$C(u) = \max \left\{ C(u_L) + C(u_R), -1 + \max_{v: (u,v) \in \mathcal{S}} O_{\text{close}}(u, v) \right\}.$$

The first term keeps the partitions below the two children separate, leaving u unused. The second closes the active component at u and charges its -1 penalty. For a leaf ℓ , $C(\ell) = -1$. The optimum is $C(r_1)$, where r_1 is the root of T_1 . Storing the maximizing choices allows the partition to be recovered. This sweep runs in $\mathcal{O}(n^2)$ time for two trees.

A.2 Extension to Multiple Trees

For $k > 2$ trees, we can solve rWMASTP in $\mathcal{O}(kn^3)$ time. Here, states are mapped to the $\mathcal{O}(n^2)$ leaf pairs (a, b) , which uniquely define a vertex tuple $(u_1, \dots, u_k)$ of their lowest common ancestors across all trees. The DP evaluates $\mathcal{O}(n^3)$ transitions of the form $(a, b) \rightarrow (a, c)$, where c is a leaf such that $\text{lca}(T_t, \{a, c\})$ is a strict ancestor of $\text{lca}(T_t, \{a, b\})$ in all trees T_t .

A.3 Fixed Edges

Cut edges cannot be used by a component, so transitions that traverse them are disabled in every tree. Forced edges are handled only through their dual weights: a transition using such an edge includes its weight in the objective, but leaving that edge unused does not make a state infeasible. In particular, forced edges do not prohibit splitting a partition or closing a component, and singleton components remain available. The master equalities (F) enforce forced-edge coverage when the generated components are combined.